

```
# You could order a pizza in the following ways:  
❷ orderPizza('large')    # large, defaults to regular, no topping  
  
    orderPizza('large', style='regular') # same as above  
  
❸ orderPizza('medium', style='deepdish', topping='mushrooms')  
  
    orderPizza('small', topping='mushrooms') # style defaults to regular
```

Listing 7-1: A function with a keyword parameter defaulting to `None`

The first and second calls would be seen as the same, with the value of the variable `topping` set to `None` ❷. In the third and fourth calls, the value of `topping` is set to `'mushrooms'` ❸. Because `'mushrooms'` is not `None`, in these calls the code would add in an extra charge for a topping on the pizzas ❶.

Using `None` as a default value for a keyword parameter gives you a way to see if the caller provided a value in the call. This may be a very subtle use of keyword parameters, but it will be very useful in our upcoming discussion.

Choosing Keywords and Default Values

Using default values makes calling functions and methods simpler, but there is a downside. Your choice of each keyword for keyword parameters is very important. Once programmers start making calls that override default values, it's very difficult to change the name of a keyword parameter because that name must be changed in *all* calls to the function or method in lockstep. Otherwise, code that was working will break. For more widely distributed code, this can potentially cause a great deal of pain to programmers using your code. Bottom line, don't change the name of a keyword parameter unless it is absolutely necessary. So, choose wisely!

It's also very important to use default values that should suit the widest possible range of users. (On a personal note, I *hate* mustard! Whenever I go to Burger King, I have to remember to specify no mustard or I'll get what I consider to be an inedible hamburger. I think they made a bad default choice.)